Deep Learning Framework for Automated Plant Disease Detection Using Leaf Images

V. Sudhir Naik, G. Akshith Reddy, K. Harish Reddy, K. Yaswanth Reddy
Department of Computer Science and Engineering
K L (Deemed to be) University, Hyderabad, India
Guide: Dr. K. Swanthana, Assistant Professor
KLH Bowrampet Campus

***Abstract*—Plant diseases can reduce crop quality and agricultural productivity. Early identification is important, but conventional visual inspection may require specialist knowledge and can be difficult to access in some agricultural regions. This paper presents PlantCare AI, an image-based plant disease screening framework that applies deep learning to leaf images. The project overview identifies EfficientNetB0 with transfer learning as the classifier for 15 supported healthy and disease categories involving tomato, potato, and pepper. The application performs image preprocessing, predicts a supported category, displays a confidence score, and provides Grad-CAM visual explanations. It also includes general plant-care recommendations, an experimental severity-estimation feature, and a Streamlit interface. The framework is intended for preliminary screening and educational decision support rather than professional diagnosis. The supplied project report contains a model-version inconsistency: its abstract describes EfficientNetB0, whereas its detailed experimental chapter reports comparisons among EfficientNetV2 variants, including an EfficientNetV2-B1 result table. Therefore, reported EfficientNetV2 metrics are presented separately and are not attributed to the EfficientNetB0 application. This paper documents the system architecture, dataset context, preprocessing, transfer-learning methodology, evaluation measures, reported experimental observations, limitations, and future work.**

***Index Terms*—Plant disease detection, leaf image classification, deep learning, convolutional neural network, EfficientNet, transfer learning, Grad-CAM, explainable artificial intelligence, PlantVillage, Streamlit.**

I. Introduction

Agriculture is essential to food security and economic development. Plant diseases caused by fungi, bacteria, viruses, and other agents can affect plant growth, crop quality, and yield. Delayed identification can allow disease symptoms to spread and may increase crop losses. Farmers commonly inspect leaves and other plant parts visually. Although visual inspection is useful, it can be time-consuming and may depend on access to agricultural expertise.

Computer vision and deep learning provide methods for learning visual patterns directly from images. Convolutional neural networks (CNNs) have been applied to image classification and plant disease recognition [1]–[3]. Public image datasets such as PlantVillage have supported the development and comparison of image-based disease classifiers [2], [7]. However, strong performance on curated images does not automatically guarantee reliable performance on field images, where illumination, backgrounds, camera quality, leaf orientation, and disease severity can vary.

PlantCare AI is designed as an accessible preliminary screening application. A user uploads a leaf image, the application preprocesses the image, and a trained classifier returns the most likely supported category with a confidence value. Grad-CAM is used to produce a visual heatmap that indicates image regions contributing to the prediction [6]. General plant-care information is then shown to the user.

A. Background

Traditional image-processing pipelines often depend on manually designed features such as color, texture, and shape. CNNs can learn hierarchical features from image data, reducing the need for manual feature construction [1], [14]. Transfer learning reuses representations learned from a large source dataset and adapts them to a target task. This is useful when the target image dataset is smaller than the datasets commonly used to train deep networks from scratch.

EfficientNet introduced compound scaling of network depth, width, and input resolution [4]. EfficientNetV2 later introduced architectural and training improvements intended to increase training speed and parameter efficiency [5]. The project report's abstract identifies EfficientNetB0 as the application model. The detailed experimental chapter, however, discusses EfficientNetV2-B1 and other EfficientNetV2 variants. This distinction is maintained throughout this paper.

B. Problem Statement

Manual plant disease identification may be slow, inconsistent, and dependent on expert availability. A digital tool is needed to support preliminary screening from leaf photographs and to present a prediction in an accessible form. The system must also communicate that its output is limited to the categories represented during model development.

C. Research Gap

Prior research has demonstrated the usefulness of CNNs and transfer learning for plant disease classification [2], [3], [9]. However, practical use requires more than a class label. Users benefit from a simple interface, prediction confidence, visual explanation, and clear limitations. PlantCare AI brings classification, Grad-CAM visualization, general plant-care guidance,

and an interactive Streamlit application into one workflow. Its present scope remains limited to 15 supported categories and should not be interpreted as a universal plant diagnostic system.

D. Objectives

The objectives of this work are:

1) To develop an image-based plant disease screening framework.
2) To classify leaf images into 15 supported healthy and disease categories involving tomato, potato, and pepper.
3) To apply transfer learning using the model identified in the project overview as EfficientNetB0.
4) To display predicted category and confidence information.
5) To provide Grad-CAM visual explanations.
6) To provide general plant-care guidance and an experimental severity estimate.
7) To deploy the prototype through a Streamlit interface.

E. Research Questions

The study is guided by the following questions:

1) Can transfer learning support classification of the supported leaf-image categories?
2) Can Grad-CAM provide a useful visual indication of image regions influencing a prediction?
3) Can a lightweight web interface make preliminary image-based screening accessible?
4) What limitations arise when moving from curated image datasets to field conditions?

F. Contributions

The project contributions are the integration of image preprocessing and deep-learning classification, confidence display, Grad-CAM visualization, general plant-care recommendations, experimental severity estimation, and Streamlit-based interaction. The paper also identifies the model-version inconsistency in the supplied report so that implementation claims and experimental figures are not conflated.

G. Organization

Section II reviews related work. Section III describes the proposed framework and mathematical formulation. Section IV presents the dataset and implementation setup. Section V summarizes the results documented in the project report. Section VI discusses implications, explainability, and limitations. Section VII concludes the paper and identifies future work.

II. Related Work

A. Traditional Image Processing

Earlier plant disease recognition systems commonly used handcrafted color, shape, and texture descriptors with conventional classifiers. Such approaches can be computationally manageable but depend on the quality of feature design and may be sensitive to variations in image acquisition. Image texture descriptors such as the gray-level co-occurrence matrix (GLCM) have also been used to characterize spatial relationships among neighboring pixels [17].

B. Machine Learning Approaches

Classifiers such as support vector machines (SVMs), random forests, and logistic regression have been used with handcrafted or extracted image features [15], [16]. These methods can perform well on selected datasets, but their performance depends on feature quality and the representativeness of the training data.

C. Deep Learning Approaches

CNNs learn multiple levels of visual representation and have become widely used in computer vision [1], [10], [11]. Mohanty *et al.* demonstrated image-based plant disease classification using deep learning and the PlantVillage dataset [2]. Reviews by Liu and Wang summarize deep-learning methods for plant disease and pest detection [3]. Too *et al.* compared transfer-learning architectures for plant disease classification [9].

D. Transfer Learning and Efficient Architectures

Transfer learning allows a network pretrained on a large image corpus to be adapted to a target classification task. MobileNet introduced depthwise separable convolutions for efficient mobile vision models [13]. DenseNet uses feature reuse through dense connectivity [12]. ResNet uses residual connections to support deeper networks [11]. EfficientNet uses compound scaling to balance depth, width, and resolution [4], while EfficientNetV2 focuses on smaller models and faster training [5]. These architectures provide relevant design choices for resource-aware agricultural image classification.

E. Datasets and Field-Image Challenges

PlantVillage is an influential public dataset for image-based plant disease research [2], [7]. PlantDoc was introduced to provide more visually challenging plant disease images collected under less controlled conditions [8]. Field imagery may contain cluttered backgrounds, partial occlusion, shadows, and variations in camera distance. Consequently, dataset composition and evaluation protocol must be reported when interpreting accuracy.

F. Explainability

A high classification score does not explain which image regions influenced a prediction. Grad-CAM uses gradients flowing into a convolutional layer to generate a class-discriminative localization map [6]. Other explanation approaches include LIME [18] and SHAP [19]. In PlantCare AI, Grad-CAM is used as a visual aid; it does not prove that the model has learned biologically valid causal features.

G. Critical Comparison

Table I provides a qualitative comparison of common approaches. The table describes typical capabilities, not measured head-to-head results from the PlantCare AI implementation.

TABLE I
QUALITATIVE COMPARISON OF PLANT DISEASE RECOGNITION APPROACHES

Approach	Feature representation	Typical strength	Typical limitation
Handcrafted image features	Color, shape, texture, GLCM	Interpretable and relatively simple	Depends on feature engineering and image conditions
Classical ML	SVM, random forest, logistic regression	Can work with compact feature vectors	Limited representation when features are insufficient
CNN from scratch	Learned convolutional features	Learns task-specific visual patterns	Requires adequate labeled data and computation
Transfer learning	Pretrained CNN backbone and task-specific head	Reuses generic visual features	Can inherit source-domain bias and dataset limitations
PlantCare AI prototype	EfficientNet-based classification, confidence display, Grad-CAM, Streamlit	Combines prediction and user-facing explanation in one workflow	Limited to supported categories; implementation/model version needs confirmation

TABLE II
DATASET INFORMATION DOCUMENTED IN THE PROJECT REPORT

Item	Reported description
Primary dataset	PlantVillage
Plant groups	Tomato, potato, pepper
Supported categories	15 healthy/disease categories
Image count in detailed experimental section	20,639
Additional dataset discussed	Drone-captured rice images
Application scope	15 supported categories only

III. PROPOSED METHODOLOGY

A. Overall Framework

The proposed application follows an image-to-prediction pipeline. The user uploads a leaf image through the Streamlit interface. The image is decoded, resized, and normalized according to the model's expected input format. The trained network extracts visual features and produces class probabilities. The category with the largest probability is selected. The application displays the predicted class and confidence value. When enabled, Grad-CAM generates a heatmap for visual inspection, after which general plant-care information and the experimental severity estimate can be displayed.

B. Dataset

The project report describes a 15-class PlantVillage subset covering tomato, potato, and bell pepper, with 20,639 images in the detailed experimental discussion. The report also refers to a separate drone-captured rice dataset in its extended experimental material. The PlantCare AI application overview, however, defines its supported scope as 15 categories involving tomato, potato, and pepper. Therefore, the rice drone dataset is not treated as part of the application's 15-class prediction interface in this paper.

The exact class names and per-class image counts should be exported from the actual dataset directory before final submission. This avoids assuming that the downloaded subset has exactly the same class distribution as another PlantVillage release.

C. Image Preprocessing

Preprocessing standardizes the input before classification. The report's detailed experimental discussion describes resizing images to 224×224 pixels, normalization, and augmentation operations such as horizontal flipping, rotation, zooming, and color variation. The exact preprocessing applied by the deployed EfficientNetB0 application should be checked against its source code and training configuration.

For an input image I, resizing can be represented as

$$I_r = \mathcal{R}(I; H, W), \tag{1}$$

where $\mathcal{R}$ is the resizing operation and $H = W = 224$ for the reported experimental setup. Pixel normalization may be expressed as

$$I_n = \frac{I_r - \mu}{\sigma}, \tag{2}$$

where μ and σ are the channel-wise mean and standard deviation used by the selected pretrained model. The actual implementation's preprocessing function must determine the values used at inference.

D. EfficientNet Transfer Learning

EfficientNet models use compound scaling to balance network depth, width, and input resolution [4]. A generic compound scaling formulation is

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}, \tag{3}$$

subject to an approximate computational constraint

$$\alpha\beta^2\gamma^2 \approx 2, \tag{4}$$

where α, β, and γ are scaling coefficients and ϕ is a user-defined scaling factor.

The project overview identifies EfficientNetB0 as the transfer-learning backbone. In a typical transfer-learning workflow, a pretrained convolutional base extracts visual features

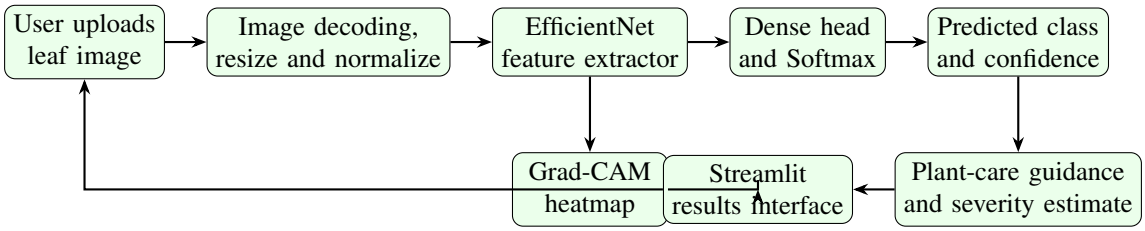


Fig. 1. PlantCare AI system architecture.

and a task-specific classification head maps the features to the target classes. The class probability for class c is computed by Softmax:

$$P(y = c \mid x) = \frac{\exp(z_c)}{\sum_{j=1}^{C} \exp(z_j)}, \tag{5}$$

where z_c is the logit for class c and $C = 15$ is the number of supported categories. The predicted category is

$$\hat{y} = \arg \max_{c \in \{1,\ldots,C\}} P(y = c \mid x). \tag{6}$$

E. Grad-CAM Explainability

Grad-CAM creates a coarse localization map using gradients of a target class score with respect to feature maps in a convolutional layer [6]. For feature map A^k, the importance weight for class c is

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}, \tag{7}$$

where Z is the number of spatial positions. The class activation map is

$$L_{\mathrm{Grad-CAM}}^c = \mathrm{ReLU}\left(\sum_k \alpha_k^c A^k \right). \tag{8}$$

The resulting heatmap is overlaid on the input leaf image. It is intended to support visual inspection, not to provide a medical-style diagnosis or guarantee that every highlighted region is biologically meaningful.

F. Severity Estimation and Recommendations

The project documentation describes an experimental severity-estimation module and general disease-prevention or treatment guidance. Since the report does not provide a validated severity ground truth protocol or a quantitative severity evaluation, severity is described as experimental. Recommendations are general information and should not be treated as verified pesticide prescriptions.

G. Streamlit Application

The user interface is implemented using Streamlit. It accepts an uploaded image and presents the predicted category, confidence score, Grad-CAM visualization, and available plant-care information. The interface makes the model accessible without requiring the user to interact directly with a Python training script.

TABLE III
SOFTWARE COMPONENTS DOCUMENTED FOR PLANTCARE AI

Component	Purpose
Python	Application and model development
TensorFlow/Keras	Deep-learning model implementation
OpenCV	Image processing support
EfficientNetB0	Backbone identified in project overview
Grad-CAM	Visual explanation
Streamlit	Interactive web application

IV. EXPERIMENTAL SETUP

A. Implementation Environment

The project report identifies Python, TensorFlow/Keras, OpenCV, and Streamlit as project technologies. Exact library versions and the hardware used by the current application should be inserted from the actual environment before publication.

B. Evaluation Metrics

For a multi-class classification task, common evaluation metrics include accuracy, precision, recall, and F1-score. Accuracy is

$$\mathrm{Accuracy} = \frac{\sum_{c=1}^{C} TP_c}{\sum_{c=1}^{C} (TP_c + FP_c + FN_c)}. \tag{9}$$

For class c, precision and recall are

$$\mathrm{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \mathrm{Recall}_c = \frac{TP_c}{TP_c + FN_c}. \tag{10}$$

The class-wise F1-score is

$$F1_c = 2 \frac{\mathrm{Precision}_c \cdot \mathrm{Recall}_c}{\mathrm{Precision}_c + \mathrm{Recall}_c}. \tag{11}$$

Macro-averaged metrics are calculated by averaging the corresponding class-wise values. These definitions are included to specify the evaluation protocol; they do not imply that every metric was measured for the deployed B0 application.

TABLE IV
REPORTED PLANTVILLAGE RESULTS FOR EFFICIENTNETV2 VARIANTS

Model	Acc. (%)	F1 (%)	Prec. (%)	Recall (%)
V2-B0	98.10	98.12	98.86	98.79
V2-B1	99.71	99.43	99.51	99.47
V2-B2	99.55	99.62	99.59	98.58
V2-B3	99.62	99.55	99.66	99.26
V2-L	99.65	99.61	99.65	99.67
V2-M	99.67	99.67	99.57	99.68
V2-S	96.93	96.93	96.87	96.78

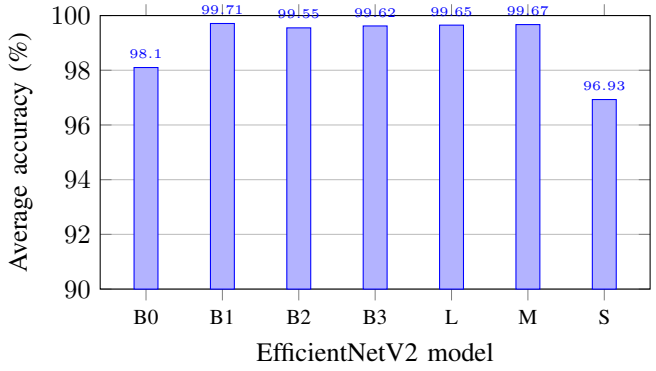


Fig. 2. Average accuracy values reported for EfficientNetV2 variants in the project documentation.

C. Model-Version and Result Provenance

The project documentation is internally inconsistent about the model version. Its abstract and project overview specify EfficientNetB0. The detailed experimental chapter describes an EfficientNetV2-B1 fine-tuning study and compares EfficientNetV2-B0, B1, B2, B3, S, M, and L. The values in Table IV and Fig. 2 are transcribed from that experimental chapter. They must not be represented as measurements of the deployed EfficientNetB0 application unless the implementation and experiment logs confirm that they belong to the same model and run.

V. RESULTS AND DISCUSSION

A. Reported EfficientNetV2 Comparison

The project report's PlantVillage comparison table reports the following average accuracies for EfficientNetV2 variants. The values are reproduced as documented and are not independently re-evaluated in this paper.

The report identifies EfficientNetV2-B1 as the highest-accuracy model in this comparison, with a reported average accuracy of 99.71%. This result belongs to the report's EfficientNetV2 experiment. The application overview separately names EfficientNetB0; therefore, additional confirmation is required before using the V2-B1 result as the deployed application's performance.

B. Additional Reported Evaluation

The report also states that the detailed EfficientNetV2-B1 experiment achieved an unseen PlantVillage test accuracy of



Fig. 3. Second-order image statistics transcribed from the project report. The large contrast scale makes the other values visually compressed.

98.93%, macro precision of 98.74%, macro recall of 98.86%, macro F1-score of 98.78%, balanced accuracy of 98.86%, Cohen's kappa of 0.9884, and AUC-ROC of 1.000. These are reported values from the supplied project documentation and are not independently verified here. The report further mentions a 10-fold result of $99.50 \pm 0.16\%$ in a separate passage. Because the report includes multiple experimental descriptions and model variants, the final paper should cite the exact experiment configuration and saved evaluation output associated with each value.

C. Dataset Statistics

The project report includes first-order and second-order image statistics. The reported first-order values include mean -0.150, variance 0.749, standard deviation 0.843, skewness -0.502, and kurtosis -0.361. The reported second-order GLCM values include contrast 6060.39, correlation 0.466, energy 0.0589, and homogeneity 0.1307.

D. Qualitative Explainability

Grad-CAM heatmaps are intended to highlight regions that influence a predicted category. The project documentation describes this feature as part of the application. A rigorous quantitative localization evaluation would require expert-annotated lesion masks or bounding regions, which are not documented for the current application. Therefore, Grad-CAM is treated as a qualitative explanation.

E. Confusion Matrix and Class-Wise Analysis

The current application report does not provide a verifiable class-wise confusion matrix for the EfficientNetB0 implementation. A confusion matrix should be generated from an independent test set using the saved model and true labels. The final submission should include the matrix, per-class precision and recall, and macro and weighted F1-scores. These values are intentionally not fabricated in this manuscript.

F. Discussion

The project demonstrates how image classification and explainability can be combined in an accessible application. Transfer learning provides a practical approach for adapting

pretrained image representations to a smaller agricultural classification task. The Streamlit interface reduces the effort required to submit an image and inspect the prediction. Grad-CAM adds a visual component that can help users examine model attention.

However, the reported high values from the separate EfficientNetV2 experiment should be interpreted in light of dataset composition, split methodology, and the model-version inconsistency. Performance on a curated dataset may be higher than performance on field images. A robust deployment study should test images collected from different farms, devices, seasons, and lighting conditions.

VI. Limitations and Responsible Use

A. Limited Class Coverage

The application supports 15 categories involving tomato, potato, and pepper. It cannot reliably identify diseases outside its supported labels.

B. Dataset Domain Gap

Curated datasets may not fully represent real agricultural environments. Backgrounds, occlusion, illumination, leaf age, and symptom severity can differ in field images.

C. Model Documentation

The project report identifies EfficientNetB0 in the abstract and EfficientNetV2 variants in the experimental chapter. The final implementation should be checked to establish the exact model architecture, weights, preprocessing function, class-index mapping, and experiment provenance.

D. Severity Estimation

The severity-estimation module is described as experimental. It should not be presented as a validated measure until compared against expert-labeled severity annotations using a defined protocol.

E. Recommendations

Plant-care information is general guidance. The application is not a substitute for professional agricultural diagnosis, local extension services, or verified pesticide instructions.

VII. Conclusion and Future Work

This paper presented PlantCare AI, a deep-learning-based preliminary plant disease screening framework using leaf images. The project overview describes EfficientNetB0 transfer learning for 15 supported healthy and disease categories involving tomato, potato, and pepper. The application combines image preprocessing, class prediction, confidence display, Grad-CAM visualization, general plant-care guidance, experimental severity estimation, and a Streamlit interface.

The supplied report also contains a separate EfficientNetV2 experimental comparison. Its reported values have been retained as a distinct experiment and are not attributed to the EfficientNetB0 application. This distinction is necessary for reproducibility and accurate reporting.

Future work includes confirming the exact deployed architecture, publishing the class mapping and dataset split, evaluating on an independent field-image dataset, reporting per-class metrics and a confusion matrix, validating severity estimation against expert annotations, and measuring inference time and memory usage. Further work can also investigate mobile deployment and lightweight model alternatives.

Declarations

Funding: No external funding information is provided in the supplied project documentation.

Conflict of Interest: The authors should confirm and update this declaration before submission.

Data Availability: The project documentation identifies PlantVillage as a dataset source. The exact downloaded version, class directory structure, and train/validation/test split should be specified in the final submission.

Ethics Statement: The system is intended for preliminary screening and educational decision support. It does not replace professional agricultural diagnosis.

Author Contributions: V. Sudhir Naik, G. Akshith Reddy, K. Harish Reddy, and K. Yaswanth Reddy contributed to the student project. Individual contribution statements should be confirmed by the team.

Acknowledgment: The authors acknowledge the guidance of Dr. K. Swanthana, Assistant Professor, KLH Bowrampet Campus, and the support of K L (Deemed to be) University.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015. [Online]. Available: https://doi.org/10.1038/nature14539

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, Art. no. 1419, 2016. [Online]. Available: https://doi.org/10.3389/fpls.2016.01419

[3] J. Liu and X. Wang, "Plant diseases and pests detection based on deep learning: A review," *Plant Methods*, vol. 17, Art. no. 22, 2021. [Online]. Available: https://doi.org/10.1186/s13007-021-00722-9

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, PMLR, vol. 97, pp. 6105–6114, 2019. [Online]. Available: https://proceedings.mlr.press/v97/tan19a.html

[5] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. ICML*, PMLR, vol. 139, pp. 10096–10106, 2021. [Online]. Available: https://proceedings.mlr.press/v139/tan21a.html

[6] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE ICCV*, pp. 618–626, 2017. [Online]. Available: https://doi.org/10.1109/ICCV.2017.74

[7] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," 2015. [Online]. Available: https://github.com/salathegroup/plantvillage_deeplearning_paper_dataset

[8] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," in *Proc. 7th ACM IKDD CoDS and 25th COMAD*, pp. 249–253, 2020. [Online]. Available: https://doi.org/10.1145/3371158.3371196

[9] E. C. K. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019. [Online]. Available: https://doi.org/10.1016/j.compag.2018.03.032

[10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, vol. 25, 2012. [Online]. Available: https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, pp. 770–778, 2016. [Online]. Available: https://doi.org/10.1109/CVPR.2016.90

[12] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE CVPR*, pp. 4700–4708, 2017. [Online]. Available: https://doi.org/10.1109/CVPR.2017.243

[13] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017. [Online]. Available: https://arxiv.org/abs/1704.04861

[14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: https://www.deeplearningbook.org/

[15] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995. [Online]. Available: https://doi.org/10.1007/BF00994018

[16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324

[17] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973. [Online]. Available: https://doi.org/10.1109/TSMC.1973.4309314

[18] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. ACM SIGKDD*, pp. 1135–1144, 2016. [Online]. Available: https://doi.org/10.1145/2939672.2939778

[19] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://papers.nips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[20] J. Deng *et al.*, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE CVPR*, pp. 248–255, 2009. [Online]. Available: https://doi.org/10.1109/CVPR.2009.5206848

[21] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, 2015. [Online]. Available: https://arxiv.org/abs/1409.1556

[22] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE CVPR*, pp. 1–9, 2015. [Online]. Available: https://doi.org/10.1109/CVPR.2015.7298594

[23] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE CVPR*, pp. 1251–1258, 2017. [Online]. Available: https://doi.org/10.1109/CVPR.2017.195

[24] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[25] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, PMLR, vol. 37, pp. 448–456, 2015. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.html

[26] N. Srivastava *et al.*, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html

[27] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, Art. no. 60, 2019. [Online]. Available: https://doi.org/10.1186/s40537-019-0197-0

[28] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[29] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org/

[30] Streamlit Inc., "Streamlit documentation," 2024. [Online]. Available: https://docs.streamlit.io/

[31] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000. [Online]. Available: https://opencv.org/

[32] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[33] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, pp. 211–252, 2015. [Online]. Available: https://doi.org/10.1007/s11263-015-0816-y

[34] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. ICLR*, 2014. [Online]. Available: https://arxiv.org/abs/1312.6114

[35] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008. [Online]. Available: https://jmlr.org/papers/v9/vandermaaten08a.html

[36] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. [Online]. Available: https://doi.org/10.1016/j.patrec.2005.10.010

[37] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. IJCAI*, pp. 1137–1143, 1995. [Online]. Available: https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf

[38] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, 2018. [Online]. Available: https://doi.org/10.1016/j.neunet.2018.07.011